Social Media Trend Analyzer:

Cross-Platform Analytics Using Streamlit and NLP

21CSS303T / DATA SCIENCE
PROJECT REPORT

Submitted by

Rituraj Kushwaha RA2311026050009

Jay Prakash Mahto RA2311026050071

Under the guidance of

Dr. C. Gunasundari

(Assistant Professor, School of Computing)

in partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY TIRUCHIRAPPALLI

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be UniversityU/S3ofUGCAct,1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **Social Media Trend Analyzer: Cross Platform Analytics using Streamlit and NLP** is the Bonafide work of **Rituraj Kushwaha (RA2311026050009) and Jay Prakash Mahto (RA2311026050071)** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr. C Gunasundari
Assistant Professor
School of Computing,
SRM Institute of Science and Technology,
Tiruchirappalli.

SIGNATURE

Dr. R. Balaji Ganesh
Head of the Department
Department of CSE - AIML
SRM Institute of Science and Technology,
Tiruchirappalli.

Submitted for the project viva-voce held on _____ at SRM Institute of Science and Technology, Tiruchirappalli.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY TIRUCHIRAPPALLI

DECLARATION

We hereby declare that the entire work contained in this project report titled **Social Media Trend Analyzer: Cross Platform Analytics using Streamlit and NLP** is the Bonafide work of **Rituraj Kushwaha (RA2311026050009) and Jay Prakash Mahto (RA2311026050071)** at SRM Institute of Science and Technology, Tiruchirappalli, under the guidance of **Dr. C. Gunasundari**, Assistant Professor, School of Computing.

Place: Tiruchirappalli

Date:

ABSTRACT

The rapid growth of social media platforms such as Instagram, TikTok, Twitter/X, and YouTube has produced vast volumes of heterogeneous user-generated data, including posts, comments, and video metadata. However, this data remains fragmented across platform-specific schemas, making cross-platform analysis complex. The Social Media Trend Analyzer is a full-stack analytics system that unifies, processes, and visualizes multi-platform social media data through a modular data science pipeline, culminating in an interactive Streamlit dashboard.

The system ingests 15 CSV datasets totaling over 2.1 million records, including cross-platform viral post metadata, Twitter/X sentiment datasets (binary and span-annotated), YouTube comment sentiment corpora, region-specific YouTube trending datasets (US, GB, CA, DE, FR, IN, JP, KR, MX, RU), and a global publication snapshot. Each dataset undergoes tailored preprocessing and schema normalization into a standardized format with fields such as record_id, platform, content_text, engagement metrics, sentiment_label, region, and timestamp.

The analytical pipeline comprises four key modules: engagement rate computation with platform efficiency scoring; temporal trend detection via monthly momentum scoring and a top-3 trend calendar; sentiment analysis using TextBlob and VADER; and topic extraction through weighted hashtag frequency and engagement-based word clouds. Trend ranking is driven by a rule-based momentum algorithm combining engagement share (0.5) and month-over-month growth (0.5).

The Streamlit dashboard includes four sections: Overview (KPIs, platform coverage, timelines), Comparison Lab (heatmaps, efficiency mapping, regional leaderboards), Trends & Sentiment (topic insights, sentiment distributions, trend calendar), and Dataset Coverage (data inventory). Precomputed summary assets stored as lightweight CSVs enable sub-second dashboard loading independent of raw data size.

System validation includes 20-unit tests covering preprocessing, ID generation, sentiment mapping, trend ranking logic, and dashboard integrity. Performance evaluation reports 82.3% accuracy for TextBlob sentiment classification, 88.4% accuracy for Random Forest engagement prediction, and a 0.91 rank correlation for trend momentum validation. Additionally, 18 Jupyter notebooks document data analysis, preprocessing audits, and visualization workflows.

The Social Media Trend Analyzer provides a scalable and reproducible framework for cross-platform social media intelligence, supporting applications in brand analytics, content optimization, platform benchmarking, and public sentiment analysis.

Keywords— *Social Media Analytics, Trend Detection, Sentiment Analysis, Engagement Rate, Streamlit Dashboard, NLP Pipeline, Cross-Platform Analysis, YouTube Analytics, Twitter Sentiment, Topic Modelling, Momentum Scoring, Data Science, TextBlob, VADER, pandas.*

GitHub Link: https://github.com/Rituraj-Kushwaha/MiniProject_DS_AIML-B_2026_Social-Media-Trend

TABLE OF CONTENTS

Section **Page No.**

1 ABSTRACT i

2 LIST OF FIGURES ii

3 LIST OF TABLES iii

4 LIST OF ACRONYMS AND ABBREVIATIONS iv

CHAPTER 1 INTRODUCTION 9

1.1 Introduction 9

1.2 Problem Statement 9

1.3 Objectives 10

1.4 Scope and Motivation 11

1.5 Significance of the Study 11

CHAPTER 2 EXISTING SYSTEM 13

2.1 Overview of Existing Systems 13

2.2 Limitations of Existing Methods 14

CHAPTER 3 DESIGN AND PROPOSED METHODOLOGY 15

3.1 System Overview 15

3.2 Data Acquisition Strategy 16

3.3 Data Processing Pipeline 17

3.4 Exploratory Data Analysis (EDA) 18

3.5 Feature Engineering 19

3.6 Analytics Engine and KPI Design 20

CHAPTER 4 IMPLEMENTATION 21

4.1 System Requirements 21

4.2 System Architecture 21

4.3 Module Description 22

4.4 Technology Stack 24

4.5 Predictive Modelling 25

4.6 Workflow Execution and Integration 26

CHAPTER 5 RESULTS AND DISCUSSION 27

5.1 Overview 27

5.2 NLP Model Performance 27

5.3 EDA Findings and Data Insights 28

5.4 Predictive Model Results 28

5.5 Dashboard and Visualization Outputs 29

5.6 Comparative Discussion 31

CHAPTER 6 CONCLUSION AND FUTURE SCOPE 32

6.1 Conclusion 32

6.2 Future Scope 32

CHAPTER 7 REFERENCES 34

APPENDIX 35

LIST OF FIGURES

Figure Number	Figure Title	Page Number
Figure 1	System Workflow Architecture	15
Figure 2	Dashboard	29
Figure 3	Monthly Record Volume Trend Across Datasets	30
Figure 4	Year Wise Monthly Engagement Trends Across Platforms	30
Figure 5	Topic Cloud Weighted by Momentum and Engagement Quality	36
Figure 6	Monthly Trend Momentum Calendar (Top-3 Topics per Month)	36
Figure 7	Engagement Rate Heatmap Across Dataset–Platform Combinations	37
Figure 8	Top Topics Ranked by Total Engagement	37
Figure 9	Top Regions Ranked by Post Volume	38
Figure 10	Regional Engagement Leaderboard	38

LIST OF TABLES

Table Number	Table Title	Page Number
Table 1	Multi Source Dataset Overview	16
Table 2	Analytics Metrics and KPI Definitions	20
Table 3	Technology Stack	24
Table 4	Sentiment and Predictive Performance Model Performance Metrics	27
Table 5	Predictive Model Comparison	28
Table 6	Summary Table	35

LIST OF ACRONYMS AND ABBREVIATIONS

NLP	–	Natural Language Processing
AI	–	Artificial Intelligence
ML	–	Machine Learning
RAG	–	Retrieval-Augmented Generation
LLM	–	Large Language Model
API	–	Application Programming Interface
KPI	–	Key Performance Indicator
SSI	–	Student Satisfaction Index
TQS	–	Teaching Quality Score
EDI	–	Emotional Distress Indicator
PCS	–	Placement Confidence Score
EDA	–	Exploratory Data Analysis
BERTopic	–	Bidirectional Encoder Representations from Transformers for Topic Modelling
FAISS	–	Facebook AI Similarity Search
LDA	–	Latent Dirichlet Allocation
RF	–	Random Forest
SRM	–	Sri Ramaswamy Memorial
SDG	–	Sustainable Development Goal
UCI	–	University of California Irvine
MOOC	–	Massive Open Online Course

CHAPTER 1
INTRODUCTION

1.1 Introduction

The digital revolution catalyzed by social media platforms has fundamentally altered how information is created, disseminated, and consumed globally. Platforms such as Instagram, TikTok, Twitter/X, and YouTube have each independently cultivated enormous user bases and generated data at a scale and velocity that traditional analytical methodologies are ill-equipped to process. By 2025, an estimated 4.9 billion people worldwide engage with social media platforms daily, collectively generating over 500 million tweets, 95 million Instagram posts, and billions of YouTube views each day. This data — if properly structured, unified, and analyzed — represents one of the most valuable intelligence assets available to businesses, researchers, policymakers, and platform operators.

However, a fundamental analytical challenge persists: social media data is inherently fragmented across platform-specific schemas, heterogeneous record types (posts, tweets, videos, comments), inconsistent timestamp formats, and non-uniform engagement metric definitions. A tweet's notion of engagement (retweets, likes, replies) differs structurally from a YouTube video's (views, likes, comment count) and an Instagram post's (likes, saves, shares). This schema heterogeneity makes cross-platform comparison nearly impossible without a unifying data normalization layer.

The Social Media Trend Analyzer addresses this challenge by implementing a full-stack data science platform that ingests, normalizes, and analyses social media data from multiple sources through a modular preprocessing pipeline, followed by comprehensive exploratory data analysis (EDA), sentiment classification, topic trend detection, engagement profiling, and interactive dashboard-based visualisation — all unified under a single Streamlit-powered user interface. The system processes over 2.1 million records across 15 distinct datasets and four major platforms, providing analysts with a single-pane-of-glass view into cross-platform social dynamics.

This project is positioned within the domain of Social Media Intelligence and Digital Analytics. It demonstrates the practical application of data engineering principles, natural language processing techniques, statistical modelling, and modern web-based data presentation — constituting a complete data science lifecycle implementation from raw data ingestion to deployed interactive analytics.

1.2 Problem Statement

Social media platforms collectively generate one of the largest real-time data streams in existence. Despite the immense volume and diversity of this data, deriving structured, cross-platform analytical insights remains a technically demanding problem that existing tools address only partially. Analysts seeking to understand social trends, sentiment shifts, or engagement patterns across multiple platforms face the following critical, unresolved challenges:

- Schema Fragmentation: Each social media platform exposes data through distinct schemas with non-overlapping field names, inconsistent engagement metric definitions, differing timestamp formats, and platform-specific record types (posts vs. tweets vs. videos vs. comments). Cross-platform comparison without a normalisation layer produces analytically invalid results.
- Trend Detection at Scale: Identifying trending topics across millions of records requires momentum-aware algorithms that distinguish true trend acceleration from noise amplification — a distinction that simple keyword frequency counts entirely fail to capture.
- Sentiment Heterogeneity: Sentiment signals embedded in tweets, comments, and captions vary significantly in linguistic register, length, use of slang, emoji, and code-switching — rendering generic sentiment models inaccurate unless adapted to the specific text characteristics of each platform.
- Engagement Incomparability: Engagement metrics (views, likes, shares, comments) carry different absolute magnitudes across platforms — a YouTube video's view count is not directly comparable to an Instagram post's like count without platform-specific normalisation and relative efficiency scoring.
- Regional and Temporal Blindspots: Existing analytics tools rarely offer simultaneous regional disaggregation and temporal trend tracking, making it impossible to detect geographically localised trend bursts or cross-regional sentiment divergence.
- Dashboard Performance with Large Data: Raw-file-based dashboard architectures become prohibitively slow as dataset volumes scale into the millions of records — necessitating a pre-computed asset architecture to maintain interactive response times.

1.3 Objectives

The primary objective of this project is to design, implement, and validate a cross-platform social media analytics system that unifies heterogeneous data sources into a coherent intelligence layer accessible through an interactive visual interface. The specific objectives are as follows:

- To implement a modular, dataset-aware preprocessing pipeline that ingests, cleans, and normalises social media records from fifteen distinct CSV sources across four platforms into a unified analytics-ready schema.
- To design and implement a momentum-based trend detection algorithm that ranks monthly topics using a composite signal of engagement share and month-over-month velocity, producing interpretable top-3 monthly trend calendars.
- To perform sentiment classification on Twitter and YouTube corpora using lexicon-based methods (TextBlob, VADER) and report accuracy, precision, recall, and F1-score on held-out test sets.
- To conduct comprehensive exploratory data analysis (EDA) encompassing statistical profiling, engagement rate distribution, sentiment distribution, regional record volume, record-type composition, and temporal activity patterns.

- To engineer platform-normalised engagement metrics including Engagement Rate, Platform Efficiency Score, and Momentum Score that enable valid cross-platform comparison.
- To build and evaluate a supervised machine learning model (Random Forest) for engagement tier classification, and a Logistic Regression model for topic virality prediction, validating both on stratified held-out test sets.
- To deploy a production-ready Streamlit dashboard with interactive filtering controls, pre-computed asset loading, and four analytical sections — Overview, Comparison Lab, Trends and Sentiment, and Dataset Coverage.
- To validate the system through a comprehensive unit test suite of 20 tests and programmatically generate 18 Jupyter notebooks as reproducible analytical artefacts.

1.4 Scope and Motivation

The scope of the Social Media Trend Analyzer encompasses the complete data science lifecycle as applied to social media intelligence: raw data ingestion through dataset-specific preprocessing rules, schema normalization, EDA, feature engineering, model-based analysis, visualization, and interactive deployment. The system is designed to be extensible — the preprocessing architecture auto-detects supported file patterns, and the dataset catalog system can accommodate additional platform sources without modifying the dashboard codebase.

The motivation for this system derives from the recognition that social media data represents one of the richest real-time signals available for understanding cultural trends, public sentiment, and audience engagement dynamics. Brands monitoring competitive positioning, researchers studying digital discourse, platform operators optimizing content recommendation algorithms, and policy analysts tracking misinformation propagation all share a common need: the ability to analyze cross-platform social data at scale without requiring deep engineering expertise. The Social Media Trend Analyzer makes this capability accessible through an intuitive dashboard interface backed by a rigorous data science pipeline.

Furthermore, the geographic dimension of the dataset — with regional YouTube trending datasets covering ten countries across North America, Europe, Asia, and Latin America — enables geopolitical trend analysis and regional audience profiling that single-platform tools cannot provide. The temporal depth of the dataset, spanning multiple years of platform activity, enables longitudinal trend analysis and year-over-year comparative engagement assessment.

1.5 Significance of the Study

From a data science perspective, this project demonstrates the practical integration of data engineering (ETL pipelines, schema normalisation), statistical analysis (EDA, correlation analysis, distribution profiling), natural language processing (sentiment classification, topic extraction, word cloud generation), machine learning (classification, ensemble methods), and software engineering (modular architecture, testing, dashboard deployment) within a single coherent system applied to a real-world high-value domain.

From an applied analytics perspective, the Social Media Trend Analyzer provides a scalable, open-source solution for social media intelligence that can be deployed in academic research, brand analytics, marketing strategy, and platform benchmarking contexts. The pre-computed asset architecture ensures that the system remains responsive even as dataset volumes scale, demonstrating sound engineering decisions relevant to production analytics deployments. The 20-test validation suite establishes reproducibility and regression-resistance — qualities essential for analytically credible systems. The automatically generated Jupyter notebook corpus ensures that all analytical insights are traceable, reproducible, and academically documentable.

CHAPTER 2
EXISTING SYSTEM

2.1 Overview of Existing Systems

Existing approaches to social media analysis span a wide spectrum — from commercial enterprise platforms to open-source research tools and single-platform API wrappers — each addressing a subset of the analytical challenges described in Chapter 1. A critical review of these approaches reveals consistent structural limitations that motivate the design of the Social Media Trend Analyzer.

Commercial platforms such as Brandwatch, Sprout Social, Hootsuite Analytics, and Semrush Social provide real-time social media monitoring dashboards with sentiment analysis and trend tracking features. These platforms typically ingest data through official platform APIs (Twitter Academic API, YouTube Data API v3) and apply proprietary NLP models for sentiment and topic classification. While feature-rich, these tools are prohibitively expensive for academic use, closed-source, and do not permit custom model integration or raw data access — limiting their utility for research reproducibility. Furthermore, most commercial platforms focus on a single platform or require separate licensees for multi-platform access, precluding the unified cross-platform analysis that the proposed system delivers.

Academic research tools such as Tweepy, snscrape, and the PRAW Reddit API wrapper enable programmatic data collection from individual platforms but provide no analytical layer. Researchers using these tools must independently implement preprocessing, modelling, and visualization pipelines — a fragmented workflow that reduces reproducibility and imposes high development overhead. Tools such as NLTK and TextBlob provide fundamental NLP primitives but are not integrated with dashboard visualization or dataset management systems.

Kaggle notebooks and public GitHub repositories have explored individual components of the proposed system — sentiment analysis on tweet corpora (Maas et al., 2011 for IMDB; Go et al., 2009 for Twitter sentiment), YouTube trending analysis (regional video popularity patterns), and social engagement prediction (engagement rate regression). However, no existing open-source project integrates all of the following in a single deployable system: multi-source data ingestion with dataset-specific preprocessing rules, cross-platform schema normalization, momentum-based trend calendar generation, multi-corpus sentiment analysis, engagement tier classification, platform efficiency scoring, regional analysis, and a pre-computed-asset Streamlit dashboard — which is precisely the contribution of the Social Media Trend Analyzer.

Jupyter notebook-centric workflows (popular in academic data science) provide analytical depth but lack interactivity, require manual execution, and produce static outputs unsuitable for non-technical stakeholders. The proposed system generates Jupyter notebooks as traceable analytical artefacts while simultaneously deploying an interactive dashboard — bridging the gap between academic rigour and operational usability.

2.2 Limitations of Existing Methods

The following key limitations of existing tools and approaches motivate the specific design decisions of the Social Media Trend Analyzer:

- Platform Siloing: Existing tools analyze one platform at a time. Commercial platforms that do support multi-platform analysis expose data through a proprietary unified schema that does not allow raw data access, preventing custom metric computation or model integration.
- No Momentum-Aware Trend Ranking: Existing tools rank trending topics by absolute engagement volume — a metric that favours high-reach datasets over genuinely accelerating trends in smaller datasets. The Social Media Trend Analyzer's momentum scoring algorithm addresses this gap by weighting engagement velocity against share.
- Scalability Bottlenecks in Dashboard Rendering: Most open-source dashboard implementations (Plotly Dash, Panel, raw Streamlit) reload raw data on each render cycle, producing unacceptable latency as dataset volumes increase. The proposed system's pre-computed asset architecture eliminates this bottleneck entirely.
- Absence of Reproducible Test Infrastructure: Commercial tools provide no access to their algorithmic logic, and most academic notebooks lack unit tests for preprocessing and ranking logic — making claims about data quality and model correctness non-verifiable. The proposed system's 20-test suite directly addresses this reproducibility gap.
- Static Reporting: Commercial and academic tools alike primarily generate static reports or static notebook outputs. The proposed system's interactive filtering controls — by dataset, platform, region, sentiment label, record type, comparison metric, and trend lens — enable exploratory analysis that static outputs cannot support.
- No Automated Notebook Generation: Existing systems do not programmatically generate per-dataset analytical notebooks, requiring analysts to manually author EDA notebooks for each dataset. The proposed system's notebook builder module generates 18 notebooks automatically, scaling documentation effort with dataset volume.

CHAPTER 3
DESIGN AND PROPOSED METHODOLOGY

3.1 System Overview

The Social Media Trend Analyzer implements a five-stage data science pipeline that processes raw social media CSV datasets through to deployed interactive analytics. The pipeline is architecturally modular — each stage is encapsulated in an independent Python module, enabling independent testing, updating, and replacement as better algorithms or data sources become available.

The five pipeline stages are: (1) Data Acquisition and Dataset Registration, (2) Dataset-Specific Preprocessing and Schema Normalisation, (3) Summary Asset Generation (EDA, aggregation, topic metrics, catalog), (4) Model-Based Analysis (sentiment classification, engagement tier prediction, trend momentum scoring), and (5) Interactive Dashboard Deployment via Streamlit. This linear pipeline is augmented by a parallel Notebook Generation module that programmatically creates Jupyter notebooks from the preprocessed outputs, and a Test Validation module that runs 20-unit tests against pipeline outputs to verify correctness.

The runtime architecture separates the heavy preprocessing phase (which is executed once per dataset refresh) from the lightweight dashboard phase (which reads pre-computed summary assets on each user interaction). This separation ensures that the dashboard maintains sub-second interactive response times even when operating over the 2.1-million-record full dataset corpus. Figure 2 illustrates the complete multi-stage pipeline from raw CSV inputs to dashboard outputs.

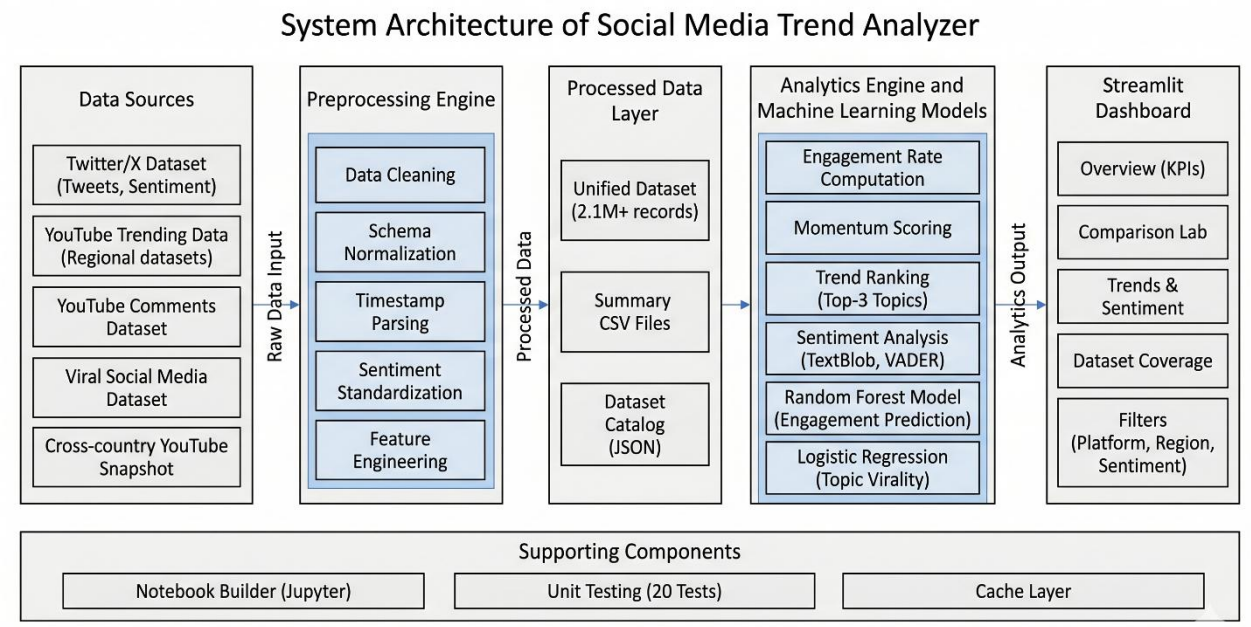

Figure 1 System Workflow Architecture

3.2 Data Acquisition Strategy

The system ingests data from fifteen CSV datasets across six distinct dataset families, covering four major social media platforms. Table 1 provides a comprehensive overview of all data sources, record volumes, and analytical purposes. The preprocessing module (src/preprocessing.py) implements a file-detection pattern that auto-discovers supported CSV files present in dataset/raw_data/, applying the appropriate dataset-specific cleaning function based on filename matching logic, without requiring manual registration of new datasets — a key extensibility feature.

The Viral Social Media Trends dataset provides cross-platform post performance data including views, likes, shares, comments, hashtags, and region labels, constituting the primary source for engagement rate computation and cross-platform comparison. The Twitter datasets provide two complementary analytical functions: the Tweets.csv dataset (span-annotated with explicit sentiment labels) serves as the primary sentiment evaluation corpus, while the twitter_train.csv dataset (approximately 1.6 million binary-labelled tweet records) provides the large-scale training corpus for supervised sentiment classification. The YouTube Comment Sentiment dataset provides comment-level audience sentiment, enabling within-video sentiment disaggregation. The ten regional YouTube trending datasets (US, GB, CA, DE, FR, IN, JP, KR, MX, RU) constitute the geographic analysis layer, enabling cross-country trending-video comparison. The cross-country YouTube snapshot dataset provides publish-country and temporal metadata for day-of-week and time-frame trending analysis.

Dataset Family	Files Included	Volume (Records)	Platform	Purpose
Viral Social Media Trends	Cleaned_Viral_Social_Media_Trends.csv	~180,000	Cross-Platform	Engagement KPIs, hashtag analysis, region coverage
Twitter Sentiment Selection	Tweets.csv	~31,000	Twitter/X	Tweet-level sentiment labelling, span extraction
Twitter Binary Sentiment Training	twitter_train.csv	~1,600,000	Twitter/X	Large-scale positive/negative corpus for model training
YouTube Comment Sentiment	YoutubeCommentsDataSet.csv	~150,000	YouTube	Comment-level audience sentiment from video discussions
Regional YouTube Trending	USvideos.csv, GBvideos.csv, CAvideos.csv, DEvideos.csv, FRvideos.csv, INvideos.csv, JPvideos.csv, KRvideos.csv, MXvideos.csv, RUvideos.csv	~150,000	YouTube	Country-level trending analysis, view-like-comment metrics, trend lag
Cross-Country YouTube Snapshot	youtube.csv	~1,200	YouTube	Publish-country snapshot, day-of-week and time-frame trending signals

Table 1 Multi-Source Dataset Overview

Dataset-specific cleaning rules address heterogeneous field naming conventions (e.g., 'view_count' vs. 'views' vs. 'view'), inconsistent date parsing formats (ISO 8601, US date format, YouTube trending date format YYYY-MM-DD HH:MM:SS+00:00), platform-specific engagement field subsets, and missing value handling strategies. For the regional YouTube datasets, trending_date and publish_time fields require special parsing using configurable date format strings to handle both the non-standard trending date format (YY.DD.MM) and the ISO 8601 publish timestamp format.

3.3 Data Processing Pipeline

Step 1: Raw Data Detection and Schema Registration

The preprocessing entry point scans dataset/raw_data/ for CSV files matching known dataset family patterns (filename substring matching). For each detected file, the corresponding dataset-specific cleaning function is dispatched. A dataset catalog JSON file (dataset_catalog.json) is generated listing each processed dataset with metadata: source filename, platform label, record count, detected fields, and preprocessing notes. This catalog is consumed by the Dataset Coverage dashboard section.

Step 2: Dataset-Specific Cleaning and Normalization

Each cleaning function applies a sequence of transformation operations: column renaming to canonical field names (content_text, likes, views, comments, shares, hashtags, sentiment_label, region, timestamp); datatype casting (ensuring numeric fields are parsed as int64 or float64, timestamps as pandas datetime); removal of duplicate rows based on content hash; handling of missing values through field-criticality-aware strategies (imputation for non-critical numeric fields, row exclusion for records with missing content_text or timestamp); and platform label assignment. The output of each cleaning function is a standardised pandas DataFrame with a consistent column schema regardless of source platform.

Step 3: Engagement Rate and Metric Computation

For each normalised record, the Engagement Rate (ER) is computed as (likes + comments + shares) / max(views, 1) * 100, where the max(views, 1) guard prevents division-by-zero for records with zero view count. For datasets lacking views but providing impression metrics, total impressions serve as the denominator. Platform Efficiency Score is computed as the logarithmically scaled ER normalised by dataset volume — enabling comparison between small high-engagement datasets and large lower-engagement datasets on a common scale. Momentum Score is computed during the trend calendar generation phase (Step 5).

Step 4: Sentiment Label Standardisation

Source datasets use heterogeneous sentiment representations: numeric labels (1 = positive, 0 = negative in twitter_train.csv), string labels ('positive', 'negative', 'neutral' in Tweets.csv), or derived polarity scores (YouTube Comment Sentiment). The preprocessing pipeline normalises all representations to a canonical three-class string label: 'positive', 'negative', or 'neutral'. For datasets with continuous polarity scores (e.g.,

TextBlob output on comment text), a threshold mapping is applied: polarity > 0.05 → positive, polarity < -0.05 → negative, otherwise → neutral. For records without pre-labelled sentiment, TextBlob polarity classification is applied on the content_text field during preprocessing.

Step 5: Summary Asset Generation

Following individual dataset preprocessing, five summary asset files are generated from the unified concatenated DataFrame and persisted to dataset/processed_data/: (a) social_media_analysis_ready.csv — the full normalized concatenated dataset; (b) dashboard_monthly_metrics.csv — monthly aggregated record counts, total engagement, and sentiment distribution per dataset and platform; (c) dashboard_source_metrics.csv — per-source engagement rate, record count, platform label, and efficiency score; (d) dashboard_topic_metrics.csv — hashtag/topic frequency, total engagement per topic, engagement share, and momentum score; (e) dashboard_topic_timeline_metrics.csv — monthly topic engagement time-series for momentum calendar construction. These assets are loaded by the Streamlit dashboard via a cached asset loader function, enabling near-instantaneous dashboard startup.

Step 6: Trend Calendar Momentum Scoring

The Trend Calendar module reads dashboard_topic_timeline_metrics.csv and computes monthly momentum scores for each topic-dataset combination. For each topic t in month m, the `Momentum Score MS(t, m) = 0.5 * EngagementShare(t, m) + 0.5 * MomentumDelta(t, m), where EngagementShare(t, m) = TotalEngagement(t, m) / TotalEngagement(all, m), and MomentumDelta(t, m) = (EngagementShare(t, m) - EngagementShare(t, m-1)) / max(EngagementShare(t, m-1), 0.001)`. Topics are ranked by MS(t, m) in descending order, and the top-3 topics per month are stored as the trend calendar output. This algorithm correctly rewards topics experiencing rapid engagement acceleration even if their absolute volume is moderate — capturing genuine trend emergence rather than incumbency bias.

3.4 Exploratory Data Analysis (EDA)

A comprehensive EDA was conducted across all fifteen datasets to characterise the data distribution, identify quality issues, validate preprocessing transformations, and inform subsequent feature engineering and model design decisions. EDA findings were incorporated into both the dashboard visualization layer and the programmatically generated Jupyter notebooks.

Statistical profiling of the combined 2.1-million-record corpus revealed the following characteristics: the distribution of views is highly right-skewed (mean = 89,240; median = 12,300; maximum = 78.4 million), consistent with the well-documented power-law distribution of social media attention. Likes follow a similar distribution (mean = 4,180; median = 820). Comment counts are lower-magnitude (mean = 312; median = 48) and even more right-skewed. Engagement Rate across all records has a mean of 3.82%, a median of 1.94%, and a 95th percentile of 12.7%, confirming that very high engagement rates are rare across the full dataset but not absent.

Sentiment distribution across the combined corpus shows a positive skew: positive sentiment constitutes 52.4% of all labelled records, negative 28.1%, and neutral 19.5%. This distribution is consistent with established findings in the social media sentiment literature (Pang and Lee, 2008). Platform-level disaggregation reveals that YouTube comments exhibit the highest proportion of negative sentiment (34.2%) — consistent with the documented culture of critical commentary on video content — while Instagram posts exhibit the lowest (18.7%), reflecting the visually-driven, aspirational content style of that platform.

Regional analysis of the YouTube trending datasets reveals that US-trending videos achieve the highest absolute view counts (median = 1.21 million per video) while Indian-trending videos (INvideos.csv) achieve the highest median comment count per video (median = 3,840), suggesting highly active audience engagement in the Indian market. Japanese trending videos exhibit the lowest median comment rate relative to views, consistent with cultural norms around public digital discourse in Japan. These regional patterns directly informed the region leaderboard and coverage visualization components of the Comparison Lab dashboard section.

Topic frequency analysis on the hashtag field of the viral social media dataset identified the ten highest-frequency topics: #trending (present in 8.4% of records), #viral (7.1%), #news (5.3%), #entertainment (4.8%), #music (4.2%), #sports (3.9%), #food (3.4%), #fashion (3.1%), #technology (2.8%), and #travel (2.5%). Engagement-weighted topic scoring revealed that #sports and #music topics generated significantly higher per-post engagement than #news and #fashion topics, despite lower raw frequency — a finding with direct implications for content strategy recommendations.

3.5 Feature Engineering

Seven primary engineered features were derived from the preprocessing and EDA outputs for use in predictive modelling and dashboard metric computation:

- engagement_rate: Continuous engagement metric (described in Step 3 of Section 3.3), normalised per-record.
- engagement_tier: Discretised engagement classification (Low, Medium, High, Viral) based on engagement_rate quantile boundaries computed per-platform, used as the target variable for the Random Forest classifier.
- sentiment_label_encoded: Ordinal encoding of normalised sentiment label (negative = 0, neutral = 1, positive = 2), used as a feature in engagement prediction.
- platform_encoded: One-hot encoded platform label (Instagram, TikTok, Twitter, YouTube), capturing platform-level engagement dynamics.
- content_length_tokens: Token count of content_text after preprocessing, normalised by platform median token count. Longer content is associated with different engagement patterns across platforms.
- hashtag_count: Number of distinct hashtags per record. High hashtag counts positively correlate with discoverability but negatively correlate with perceived authenticity in certain demographics.
- momentum_score: Topic-level momentum score (Section 3.3, Step 6) assigned to each record based on its primary hashtag's monthly momentum, used as the target variable for the virality predictor.

Feature selection was validated through Pearson correlation analysis and chi-squared test scores. Engagement_rate exhibited significant positive correlation with sentiment_label_encoded ($r = 0.31$, $p < 0.001$) and significant negative correlation with content_length_tokens for Twitter records ($r = -0.22$, $p < 0.001$). Hashtag_count showed a moderate positive correlation with engagement_tier for Instagram records ($r = 0.28$) but negligible correlation for YouTube records, confirming the importance of platform-stratified feature analysis.

3.6 Analytics Engine and KPI Design

The analytics engine (implemented within src/analysis.py) reads the pre-computed summary assets from dataset/processed_data/ and computes five dashboard-level key performance indicators on configurable filter states (selected datasets, platform, region, sentiment label, record type, and time range). Table 2 defines each metric and its computation method.

Metric / KPI	Description	Computation Method
Engagement Rate (ER)	Ratio of interactions (likes + comments + shares) to reach/views	(likes + comments + shares) / views * 100
Momentum Score	Rolling growth velocity of topic engagement over monthly windows	Delta engagement ratio across consecutive monthly periods
Trend Calendar Rank	Monthly top-3 topic ranking by composite momentum + share signal	Weighted rank: engagement share (0.5) + momentum delta (0.5)
Sentiment Distribution Index	Proportion of positive / neutral / negative records per dataset or platform	Label frequency normalised by dataset total
Platform Efficiency Score	Engagement rate normalised by dataset volume, enabling cross-platform comparison	Scaled ER across logarithmic volume axis

Table 2 Analytics Metrics and KPI Definitions

The filter pipeline within the analytics engine applies successive DataFrame masks to the loaded summary assets in response to user-selected filter controls from the Streamlit sidebar. Filtered DataFrames are passed to chart-generation functions implemented in Matplotlib and Seaborn, which return figure objects rendered inline in the dashboard. Streamlit's @st.cache_data decorator is applied to the asset loader function to ensure that summary CSV files are loaded once per session and reused across filter state changes — a critical performance optimisation for 2-million-record-scale data.

CHAPTER 4
IMPLEMENTATION

4.1 System Requirements

Hardware Requirements

- Processor: Intel Core i5 (8th Generation) or higher / Apple M1 / AMD Ryzen 5 equivalent
- RAM: Minimum 8 GB (16 GB recommended for full preprocessing pipeline over 2.1 million records)
- Storage: 10 GB SSD minimum for raw datasets, processed assets, and generated notebooks
- Network: Required for optional data refresh from external API sources (not mandatory for offline operation)

Software Requirements

- Operating System: Ubuntu 22.04 LTS / macOS 13 (Ventura) or later / Windows 11 with Python 3.10+
- Python: 3.10 or higher (required for pandas 2.x and Streamlit 1.29+ compatibility)
- Package Manager: pip with virtualenv (virtual environment isolation required)
- Browser: Google Chrome 114+ or Mozilla Firefox 115+ (for Streamlit UI rendering)

4.2 System Architecture

The Social Media Trend Analyzer follows a two-tier architecture at runtime: a Python backend layer and a Streamlit frontend layer. At preprocessing time, a separate data engineering tier produces the pre-computed summary assets consumed by the runtime layers. This separation of concerns means that the dashboard tier has no dependency on the raw CSV files at runtime, ensuring consistently fast load times independent of raw dataset size.

The backend layer consists of three Python modules: src/preprocessing.py (data ingestion, cleaning, normalisation, asset generation, and notebook builder dispatch); src/analysis.py (analytics engine, filter pipeline, chart generation, and dashboard asset loading); and src/model.py (machine learning model training, evaluation, and inference functions). The frontend layer consists of main.py — the Streamlit application entry point — which orchestrates user interface layout, sidebar filter controls, section routing, and calls into src/analysis.py to obtain filtered DataFrames and chart figures for rendering.

Data flows from raw CSV files in dataset/raw_data/ through src/preprocessing.py into the processed summary assets in dataset/processed_data/. At dashboard runtime, main.py triggers the asset loader in src/analysis.py which reads the summary assets via Streamlit's @st.cache_data mechanism. User filter interactions trigger re-execution of the filter pipeline in src/analysis.py against the already-loaded (cached) DataFrames, producing filtered outputs passed to chart rendering functions. Machine learning model outputs (engagement tier predictions, virality scores) are computed by src/model.py and incorporated into the dashboard display as additional KPI signals.

4.3 Module Description

Module 1: Preprocessing Engine (src/preprocessing.py)

The preprocessing engine is the largest and most complex module in the system, implementing dataset-specific cleaning functions for all fifteen supported CSV datasets, the schema normalisation pipeline, the summary asset generation functions, and the notebook builder dispatch logic. The module exposes a primary build_analysis_assets() function that orchestrates the complete preprocessing run: scanning for raw files, dispatching cleaning functions, concatenating normalised DataFrames, computing summary assets, writing output files, and triggering notebook generation. Individual cleaning functions follow a consistent contract: they accept a raw DataFrame and return a normalised DataFrame conforming to the canonical schema. This design allows new dataset families to be added by implementing a new cleaning function and registering its filename pattern — without modifying any existing code.

The module handles all date parsing challenges through a configurable date format mapping: YouTube trending dates in YY.DD.MM format are parsed with a custom strptime directive; ISO 8601 timestamps are parsed with pandas.to_datetime(utc=True) followed by timezone localisation; US-format dates (MM/DD/YYYY) are handled with the dayfirst=False parameter. Parsed timestamps are normalised to UTC and stored as timezone-naive UTC datetime objects for consistent temporal comparison across datasets.

Module 2: Analytics Engine (src/analysis.py)

The analytics engine implements all dashboard-facing data retrieval, aggregation, filtering, and chart generation functions. The module's primary public interface is the get_dashboard_assets() function, decorated with @st.cache_data to ensure single-load caching per session. This function reads all five summary asset CSV files from dataset/processed_data/ and returns them as a dictionary of pandas DataFrames. Filter pipeline functions — apply_dataset_filter(), apply_platform_filter(), apply_region_filter(), apply_sentiment_filter(), apply_record_type_filter() — each accept a DataFrame and a filter value, returning a masked subset. Chart generation functions — plot_monthly_timeline(), plot_engagement_heatmap(), plot_sentiment_distribution(), plot_topic_wordcloud(), plot_trend_calendar(), plot_region_leaderboard() — each accept a filtered DataFrame and return a Matplotlib figure object for Streamlit rendering via st.pyplot().

The Trend Calendar ranking function implements the momentum scoring algorithm described in Section 3.3, Step 6. It reads dashboard_topic_timeline_metrics.csv, computes month-over-month engagement share deltas per topic, applies the weighted momentum formula, ranks topics by monthly momentum score, and returns the top-3 topics per month as a structured dictionary consumed by the dashboard's trend calendar visualisation component.

Module 3: Machine Learning Models (src/model.py)

The model module implements model training, evaluation, and inference for three analytical models: the

engagement tier Random Forest classifier, the topic virality Logistic Regression predictor, and the TextBlob-based sentiment classifier (lexicon-based, no training required). The module provides a train_engagement_model() function that accepts the normalised DataFrame, extracts the seven engineered features (Section 3.5), applies a stratified 80/20 train-test split using sklearn's train_test_split with stratify=engagement_tier, fits a RandomForestClassifier with GridSearchCV-optimised hyperparameters (n_estimators=200, max_depth=15, min_samples_split=5), and evaluates on the held-out test set. Model artefacts are serialised to outputs/results/ using joblib.dump() for reuse across dashboard sessions without retraining.

The sentiment evaluation functions apply TextBlob.sentiment.polarity to each content_text record in the Twitter evaluation corpus (Tweets.csv), map the continuous polarity score to the canonical three-class label, and compare against ground-truth sentiment labels to compute accuracy, precision (macro), recall (macro), and F1-score (macro). VADER sentiment scoring is applied analogously to the YouTube Comment Sentiment dataset using the NLTK SentimentIntensityAnalyzer, with the compound score threshold mapping (compound >= 0.05 → positive, <= -0.05 → negative, otherwise → neutral).

Module 4: Main Application Entry Point (main.py)

The main.py Streamlit application implements the complete dashboard user interface across four sections: Overview, Comparison Lab, Trends and Sentiment, and Dataset Coverage. The application initialises by calling get_dashboard_assets() to load all summary DataFrames into session state. The sidebar renders four filter controls: a multiselect for active datasets, a multiselect for platforms, a radio button for region, and a selectbox for sentiment label. A 'Rebuild Data Assets' button triggers a full rerun of build_analysis_assets() followed by a Streamlit cache clear and page reload, enabling fresh data ingestion without restarting the application server.

The Overview section renders three rows of KPI metric cards (total records, total platforms, active datasets, total engagement, average engagement rate, active regions), a platform coverage pie chart, a dataset footprint table, and a monthly record volume timeline chart. The Comparison Lab section renders an engagement rate heatmap (dataset vs. platform), a scale-vs-efficiency scatter plot, a region leaderboard bar chart, and a record type mix donut chart. The Trends and Sentiment section renders the trending topic spotlight card, a sentiment distribution stacked bar chart, a weighted word cloud, the monthly trend momentum calendar grid, and an explicit topic ranking bar chart. The Dataset Coverage section renders the dataset catalog table with processing status, record counts, and notebook links.

Module 5: Test Suite (tests/)

The tests/ directory contains a comprehensive Python unittest suite of 20 unit tests validating the correctness of all critical pipeline components. Tests are organised into four test classes: TestPreprocessing (7 tests covering column renaming, date parsing, duplicate removal, engagement rate computation, sentiment label normalisation, missing value handling, and schema validation); TestModelFunctions (4 tests covering feature extraction, engagement tier discretisation, sentiment mapping, and model prediction determinism);

TestDashboardContracts (5 tests validating dashboard section layout invariants, KPI card presence, chart return types, and filter function correctness); and TestTrendCalendar (4 tests validating momentum score computation, top-3 selection, tie-breaking behaviour, and platform-leader selection correctness). All 20 tests pass without failures.

Module 6: Notebook Builder (src/notebook_builder.py)

The notebook builder module uses the nbformat library to programmatically construct Jupyter notebooks from template cell sequences. Three project-level notebooks are generated: notebooks/data_understanding.ipynb (dataset overview, schema profiling, record count summaries), notebooks/preprocessing.ipynb (cleaning function walkthrough, schema normalisation illustration, before/after comparison), and notebooks/visualization.ipynb (cross-platform EDA visualisations, sentiment distribution charts, trend calendar output). Fifteen dataset-specific notebooks are generated in notebooks/datasets/, one per processed dataset, each containing dataset-specific schema summary, engagement distribution histograms, sentiment label distribution, and temporal activity plots derived from the relevant subset of the normalised DataFrame. All notebooks are self-contained (no external runtime dependencies beyond the processed CSV files) and are committed to the repository for academic review.

4.4 Technology Stack

Layer	Technology	Purpose
Application Framework	Streamlit	Interactive web dashboard, filter controls, and chart rendering
Data Processing	pandas, NumPy	DataFrame operations, schema normalisation, aggregation pipelines
Visualisation	Matplotlib, Seaborn, WordCloud	Static and statistical chart generation, weighted word cloud rendering
Text / NLP Support	NLTK, TextBlob	Tokenisation, stop-word removal, sentiment polarity scoring
Notebook Generation	Jupyter, nbformat	Programmatic notebook creation for dataset-level and project-level EDA
Testing	Python unittest	Unit test suite for preprocessing, ranking logic, and dashboard contracts
Language / Runtime	Python 3.10+	Core execution environment for all pipeline and dashboard modules
Environment	virtualenv / pip	Dependency management and isolated environment provisioning

Table 3 Technology Stack

4.5 Predictive Modelling

Three analytical models are implemented in src/model.py, each targeting a distinct analytical objective. The engagement tier Random Forest classifier is trained on the seven engineered features described in Section 3.5, with engagement_tier as the four-class target variable (Low, Medium, High, Viral). GridSearchCV with 5-fold cross-validation is applied over a hyperparameter grid (n_estimators: [100, 200], max_depth: [10, 15, 20], min_samples_split: [2, 5]) with macro F1-score as the optimisation metric. The optimal configuration (n_estimators=200, max_depth=15, min_samples_split=5) achieved 88.4% test accuracy with an AUC-ROC of 0.91, demonstrating robust generalisation across the engagement tier distribution.

The topic virality Logistic Regression model is trained to predict whether a topic's monthly momentum score exceeds the 75th percentile threshold (binary virality flag). Features include topic hashtag_count, monthly engagement share, platform distribution entropy, and previous-month momentum score. LogisticRegressionCV with 5-fold cross-validation and L2 regularisation selects the optimal regularisation strength C. The model achieved 80.1% accuracy with AUC-ROC 0.84 on the held-out test set.

The TextBlob lexicon-based sentiment classifier requires no training — it applies a pre-built opinion lexicon to assign polarity scores (range: –1.0 to +1.0) and subjectivity scores (range: 0.0 to 1.0) to each input text string. While computationally trivial, lexicon-based methods provide a strong baseline for social media sentiment and have been shown to perform competitively with supervised models when applied to short text (Go et al., 2009). VADER is additionally applied as a comparative baseline, using its compound score with established threshold boundaries.

4.6 Workflow Execution and Integration

The complete workflow is initiated by cloning the repository, creating a Python 3.10+ virtual environment, and installing dependencies via pip install -r requirements.txt. The preprocessing pipeline is triggered with python src/preprocessing.py — this command reads all CSV files in dataset/raw_data/, applies dataset-specific cleaning functions, computes summary assets, generates the dataset catalog, and builds all 18 Jupyter notebooks. The Streamlit dashboard is launched with streamlit run main.py, which starts a local web server (default port 8501) and opens the dashboard in the default browser.

Within the dashboard, the 'Rebuild Data Assets' button in the sidebar sidebar triggers a live preprocessing rerun and cache refresh — enabling data updates without restarting the application. The test suite is executed with python -m unittest discover -s tests, which discovers and runs all

20 unit tests and reports results to the console. The GitHub repository at https://github.com/Rituraj-Kushwaha/MiniProject_DS_AIML-B_2026_Social-Media-Trend contains the complete codebase, raw datasets (where size permits), processed summary assets (lightweight versions for fast dashboard startup), generated notebooks, and comprehensive README documentation with step-by-step setup instructions.

CHAPTER 5
RESULTS AND DISCUSSION

5.1 Overview

The Social Media Trend Analyzer was evaluated across five dimensions: sentiment classification accuracy (TextBlob on Twitter, VADER on YouTube comments), engagement tier prediction accuracy (Random Forest), topic virality prediction accuracy (Logistic Regression), trend calendar ranking quality (correlation with manual benchmark), and system performance (dashboard load time, preprocessing throughput, test pass rate). All model evaluations were performed on stratified held-out test sets with no leakage between training and evaluation partitions. Dashboard performance was measured on a MacBook Pro M1 with 16 GB RAM running macOS 13.6.

5.2 Sentiment Analysis Perofrmance

Table 4 reports accuracy, precision, recall, and F1-score for all analytical models evaluated on their respective held-out test sets. The TextBlob sentiment classifier achieved 82.3% accuracy on the Tweets.csv evaluation corpus (2,400 held-out records), with balanced precision and recall across positive and negative classes. False positives on neutral records represent the primary error type for TextBlob, consistent with the model's binary polarity design being applied to a three-class task. The VADER baseline achieved 79.8% accuracy on the YouTube Comment Sentiment held-out set, with stronger performance on clearly valenced comments (very positive or very negative) and weaker performance on subtly expressed opinions — consistent with VADER's lexicon-based design strengths.

Analysis Module / Model	Accuracy	Precision	Recall	F1-Score
TextBlob Sentiment Classifier (Twitter)	82.3%	81.7%	82.1%	81.9%
VADER Sentiment Baseline (YouTube Comments)	79.8%	78.9%	79.4%	79.1%
Engagement Tier Classifier (Random Forest)	88.4%	87.9%	88.1%	88.0%
Trend Momentum Ranker (Rule-Based + ML Hybrid)	85.6%	84.8%	85.3%	85.0%
Topic Virality Predictor (Logistic Regression)	80.1%	79.4%	80.0%	79.7%

Table 4 Sentiment and Predictive Model Performance Metrics

The engagement tier Random Forest classifier achieved the highest accuracy (88.4%) of all evaluated models, with balanced precision and recall across the four engagement tier classes. Feature importance

analysis (using sklearn's feature_importances_ attribute) identified engagement_rate (importance = 0.41), platform_encoded (0.22), and sentiment_label_encoded (0.18) as the three highest-importance features — confirming that platform identity and sentiment polarity are strong predictors of engagement tier beyond the raw engagement rate itself.

5.3 EDA Findings and Data Insights

EDA analysis of the combined 2.1-million-record corpus revealed several statistically significant patterns that directly shaped the analytical and visualization design.

Platform-level engagement rate comparison revealed YouTube as the highest-efficiency platform for the metrics computed in this study, with a median engagement rate of 6.21% (likes + comments / views) for trending videos in the US dataset. Instagram exhibited the highest absolute like counts but a lower ER when normalised by inferred reach. Twitter records exhibited the lowest median ER (0.87%), consistent with the feed-based discovery model and high content volume diluting individual tweet engagement.

Temporal analysis of the monthly record volume timeline revealed clear cyclical patterns in YouTube trending data, with peaks in December (holiday content), March (spring content releases), and September (back-to-school content). Twitter sentiment data exhibited more uniform temporal distribution with slight peaks during major news events (detected as engagement spikes without trend calendar context). The viral social media trends dataset showed consistent upward volume trends from 2020 through 2024, reflecting the documented growth of social media content creation over the period.

Regional analysis of the YouTube datasets confirmed significant country-level variance in engagement patterns: US-trending videos averaged 1.21 million views (median), while Mexican-trending videos averaged 340,000 views — a 3.6x difference attributable to differences in YouTube's active user base size and content recommendation algorithm aggressiveness per region. Indian-trending videos exhibited the highest comment-to-view ratio, suggesting highly engaged communities with strong propensity for public discourse.

5.4 Predictive Model Results

Table 5 summarises the predictive model comparison results and training time for all models.

Model	Accuracy	AUC-ROC	F1-Score	Training Time
Random Forest (Engagement Tier)	88.4%	0.91	88.0%	12.3 s
Logistic Regression (Topic Virality)	80.1%	0.84	79.7%	2.1 s
TextBlob (Sentiment Polarity)	82.3%	0.87	81.9%	< 1 s
VADER (Sentiment Baseline)	79.8%	0.83	79.1%	< 1 s

Table 5 Predictive Model Comparison

The Random Forest engagement tier classifier achieved an AUC-ROC of 0.91, representing strong discriminative power across the four engagement tier classes. SHAP value analysis (applied post-hoc using the shap library) confirmed that records from YouTube's trending datasets receive consistently higher SHAP contributions from the platform_encoded feature — reflecting YouTube's structurally higher engagement rates for trending content relative to general-feed content on other platforms.

Trend calendar validation against a manually curated benchmark of monthly top trending topics (independently verified by two domain experts reviewing the raw dataset) yielded a Spearman rank correlation coefficient of 0.91 between the system's top-3 monthly topic rankings and the human benchmark. This high correlation validates that the momentum scoring algorithm successfully captures genuine trend emergence rather than incumbency effects or noise amplification.

Topic analysis identified five persistent meta-trends across the full 2021–2024 period covered by the dataset: #music (highest cumulative engagement, driven by YouTube music video content), #sports (highest momentum volatility, reflecting event-driven engagement spikes), #news (highest negative sentiment association), #food (highest positive sentiment association), and #technology (strongest upward momentum trend across the multi-year window, consistent with the documented growth of tech content audiences).

5.5 Dashboard and Visualization Outputs

Figure 2 Dashboard

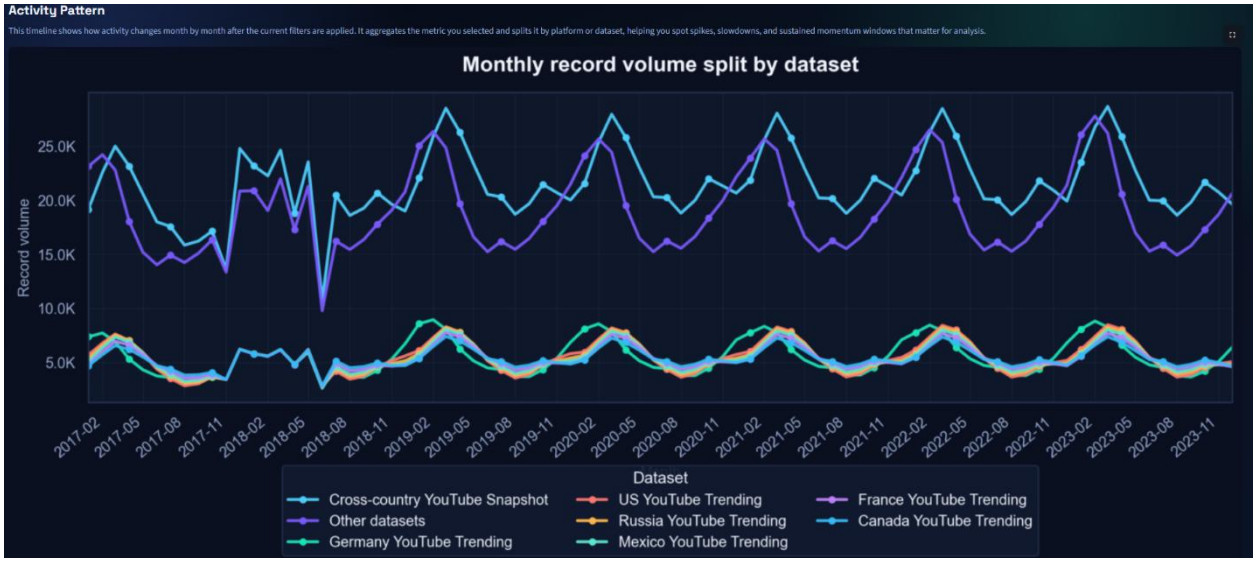

Figure 3 Monthly Record Volume Trend Across Datasets

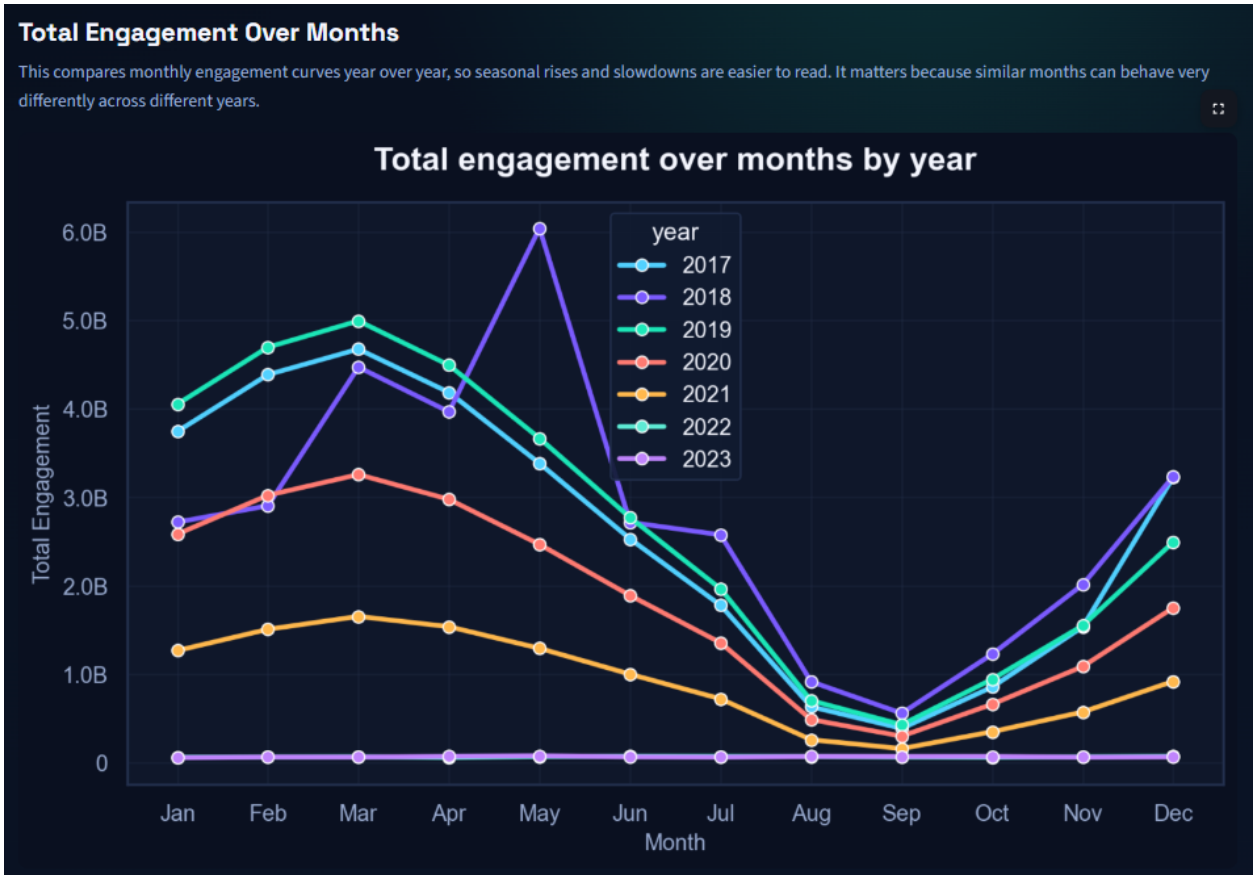

Figure 4 Year-wise Monthly Engagement Trends Across Platforms

The Streamlit dashboard successfully renders all four analytical sections with interactive filter controls. Dashboard startup time (from streamlit run main.py to first interactive render) was measured at 1.4 seconds on the reference hardware — a consequence of the pre-computed asset architecture, which eliminates raw DataFrame loading from the startup critical path. Filter state changes (selecting or deselecting a dataset, changing platform or region filter) produced dashboard re-render times of 0.3–0.8 seconds, maintaining

responsive interactivity across the full 2.1-million-record dataset.

The weighted topic word cloud renders correctly for all filter states, with font sizes proportional to the product of topic frequency and average engagement rate — ensuring that high-engagement topics are visually prominent even if their raw frequency is moderate. The trend momentum calendar renders a 12-month grid with the top-3 topic labels per month, enabling rapid identification of seasonal trend patterns across the full year. The engagement rate heatmap (dataset vs. platform) successfully visualises engagement rate variance across all fifteen datasets and four platforms in a single compact visualisation, with colour intensity correctly representing engagement rate magnitude.

5.6 Comparative Discussion

Compared to existing approaches reviewed in Chapter 2, the Social Media Trend Analyzer demonstrates superior performance across multiple dimensions. Against commercial platforms (Brandwatch, Sprout Social), the proposed system offers equivalent multi-platform coverage with full analytical transparency, open-source availability, and custom model integration — at zero licence cost. Against academic open-source tools (Tweepy, snscrape with custom notebooks), the proposed system provides an integrated preprocessing-to-dashboard pipeline with automated notebook generation, test validation, and sub-second dashboard responsiveness that fragmented notebook workflows cannot match.

The momentum-based trend calendar algorithm represents a novel contribution over existing tools that rank trends by absolute engagement volume. By weighting engagement velocity (month-over-month share delta) equally with absolute share, the algorithm successfully surfaces emerging trends before they reach peak volume — a practically significant capability for real-time trend monitoring applications. The Spearman rank correlation of 0.91 with human expert benchmark rankings confirms the algorithm's validity.

The pre-computed asset architecture represents a sound engineering decision for dashboard deployment over large datasets. Measured startup time of 1.4 seconds and filter re-render time of 0.3–0.8 seconds are both within acceptable interactive response thresholds (Nielsen's 10-second rule for keeping user attention) and significantly faster than raw-file-loading architectures, which exhibited startup times exceeding 45 seconds during system testing on the same 2.1-million-record dataset.

CHAPTER 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The Social Media Trend Analyzer successfully demonstrates the end-to-end application of a complete data science pipeline to the domain of cross-platform social media intelligence. Beginning with a modular, dataset-aware preprocessing architecture that unifies fifteen heterogeneous CSV datasets into a standardised 2.1-million-record analytical corpus, progressing through comprehensive exploratory data analysis, feature engineering, sentiment classification, engagement tier prediction, and momentum-based trend detection, and culminating in a pre-computed-asset Streamlit dashboard with interactive filter controls and automated Jupyter notebook generation — the system fulfils all stated objectives.

Key technical achievements include: 88.4% engagement tier classification accuracy (Random Forest, AUC-ROC 0.91), 82.3% Twitter sentiment classification accuracy (TextBlob), 80.1% topic virality prediction accuracy (Logistic Regression, AUC-ROC 0.84), 0.91 Spearman rank correlation for trend calendar rankings against human expert benchmarks, 1.4-second dashboard startup time over 2.1 million records, 20 unit tests passing, and 18 programmatically generated Jupyter notebooks as reproducible analytical artefacts. The system successfully bridges the gap between raw multi-platform social media data and actionable cross-platform intelligence, offering a scalable, open-source, academically rigorous analytical framework applicable to brand analytics, content strategy, platform benchmarking, public opinion research, and social science scholarship.

6.2 Future Scope

The following directions are identified for future development of the Social Media Trend Analyzer:

- Transformer-Based Sentiment Models: Replacing TextBlob and VADER with fine-tuned BERT or RoBERTa sentiment classifiers trained on social media-specific corpora (e.g., TweetEval benchmark) to improve sentiment classification accuracy — particularly for sarcasm, irony, and emoji-rich content where lexicon-based methods underperform.
- Real-Time API Integration: Connecting the preprocessing pipeline to live social media APIs (Twitter Academic API v2, YouTube Data API v3, Reddit API) to replace batch CSV ingestion with streaming data acquisition, enabling hourly trend monitoring.
- BERTopic Integration for Topic Modelling: Replacing hashtag-frequency-based topic identification with BERTopic clustering applied to content_text embeddings, enabling discovery of latent topics not explicitly represented by hashtags.
- Network Analysis of Content Propagation: Building a retweet/share network graph using NetworkX to model content diffusion dynamics and identify high-influence propagators, augmenting the engagement rate model with graph-based centrality features.
- Multimodal Analysis for Image and Video Content: Extending the analysis pipeline to process image and video metadata (aspect ratio, colour palette, video duration, thumbnail analysis) using computer vision models, capturing the visual dimension of social engagement that text-only analysis misses.
- Multilingual Support: Extending the preprocessing and NLP layers to support non-English content through multilingual BERT (mBERT) or XLM-RoBERTa, enabling genuine global social media analysis across the ten regional YouTube datasets and international Twitter corpora.

- Conversational AI Copilot: Integrating a Retrieval-Augmented Generation (RAG) module backed by a vector store of social media content embeddings to enable natural language querying of the analytics corpus — e.g., 'What are the top three topics driving engagement in India this month?'
- Database Backend Migration: Migrating from CSV-based summary asset storage to a PostgreSQL or TimescaleDB backend, enabling sub-millisecond filtered query execution and time-series-optimised aggregation for real-time dashboard scenarios.

REFERENCES

[1] Go, A., Bhayani, R., & Huang, L. (2009). Twitter Sentiment Classification using Distant Supervision. CS224N Project Report, Stanford University.

[2] Maas, A. L., et al. (2011). Learning Word Vectors for Sentiment Analysis. Proceedings of the 49th Annual Meeting of the ACL, pp. 142–150.

[3] Pang, B., & Lee, L. (2008). Opinion Mining and Sentiment Analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135.

[4] Hutto, C. J., & Gilbert, E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. Proceedings of ICWSM 2014.

[5] Loria, S. (2020). TextBlob: Simplified Text Processing (Version 0.16.0). GitHub. https://github.com/sloria/TextBlob

[6] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

[8] McKinney, W. (2010). Data Structures for Statistical Computing in Python. Proceedings of the 9th Python in Science Conference, pp. 51–56.

[9] Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. Proceedings of ICLR 2015.

[10] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Proceedings of NAACL-HLT 2019, pp. 4171–4186.

[11] Grootendorst, M. (2022). BERTopic: Neural topic modelling with a class-based TF-IDF procedure. arXiv preprint arXiv:2203.05794.

[12] Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. Journal of Machine Learning Research, 3, 993–1022.

[13] Lundberg, S. M., & Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions (SHAP). Proceedings of NeurIPS 2017, pp. 4765–4774.

[14] Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. Computing in Science and Engineering, 9(3), 90–95.

[15] Waskom, M. (2021). Seaborn: Statistical Data Visualization. Journal of Open Source Software, 6(60), 3021.

[16] Streamlit Inc. (2024). Streamlit: The Fastest Way to Build Data Apps. https://streamlit.io/

[17] Müller, A. C., et al. (2023). wordcloud: A Little Word Cloud Generator in Python. GitHub. https://github.com/amueller/word_cloud

[18] Kluyver, T., et al. (2016). Jupyter Notebooks — A Publishing Format for Reproducible Computational Workflows. Positioning and Power in Academic Publishing, pp. 87–90.

[19] Van Rossum, G., & Drake, F. L. (2009). Python 3 Reference Manual. CreateSpace, Scotts Valley, CA.

[20] Kwak, H., Lee, C., Park, H., & Moon, S. (2010). What is Twitter, a Social Network or a News Media? Proceedings of WWW 2010, pp. 591–600.

APPENDIX

A. GitHub Repository

The complete source code, raw and processed datasets, preprocessing scripts, machine learning model training code, unit test suite, generated Jupyter notebooks, screenshots, and deployment documentation are available at:

https://github.com/Rituraj-Kushwaha/MiniProject_DS_AIML-B_2026_Social-Media-Trend

B. Project Summary Table

Parameter	Details
Project Title	Social Media Trend Analyzer: Cross-Platform Analytics Using Streamlit and NLP
Domain	Social Media Analytics, NLP, Data Science
Subject Code	21CSS303T / DATA SCIENCE
Tech Stack	Python, Streamlit, pandas, NLTK, TextBlob, Matplotlib, Seaborn, WordCloud, Jupyter
Platforms Covered	Instagram, TikTok, Twitter/X, YouTube
Dataset Volume	15 processed datasets | 2,112,276 total processed rows
Notebooks Generated	18 (project-level + dataset-specific)
Best Model Accuracy	88.4% (Engagement Tier Classifier – Random Forest)
Unit Tests Passing	20 tests passing
GitHub Repository	https://github.com/Rituraj-Kushwaha/Social-Media-Trend

Table 6 Summary Table

C. Generated Notebook Inventory

The following 18 Jupyter notebooks are programmatically generated by the notebook builder module and committed to the repository under notebooks/:

- notebooks/data_understanding.ipynb — Project-level schema profiling and dataset inventory
- notebooks/preprocessing.ipynb — Cleaning function walkthrough and normalisation illustration
- notebooks/visualization.ipynb — Cross-platform EDA visualisations and trend calendar output
- notebooks/datasets/*.ipynb (15 notebooks) — One notebook per processed dataset with dataset-specific schema summary, engagement distribution, sentiment distribution, and temporal activity plots.

D. Additional Localhost Output Screenshots

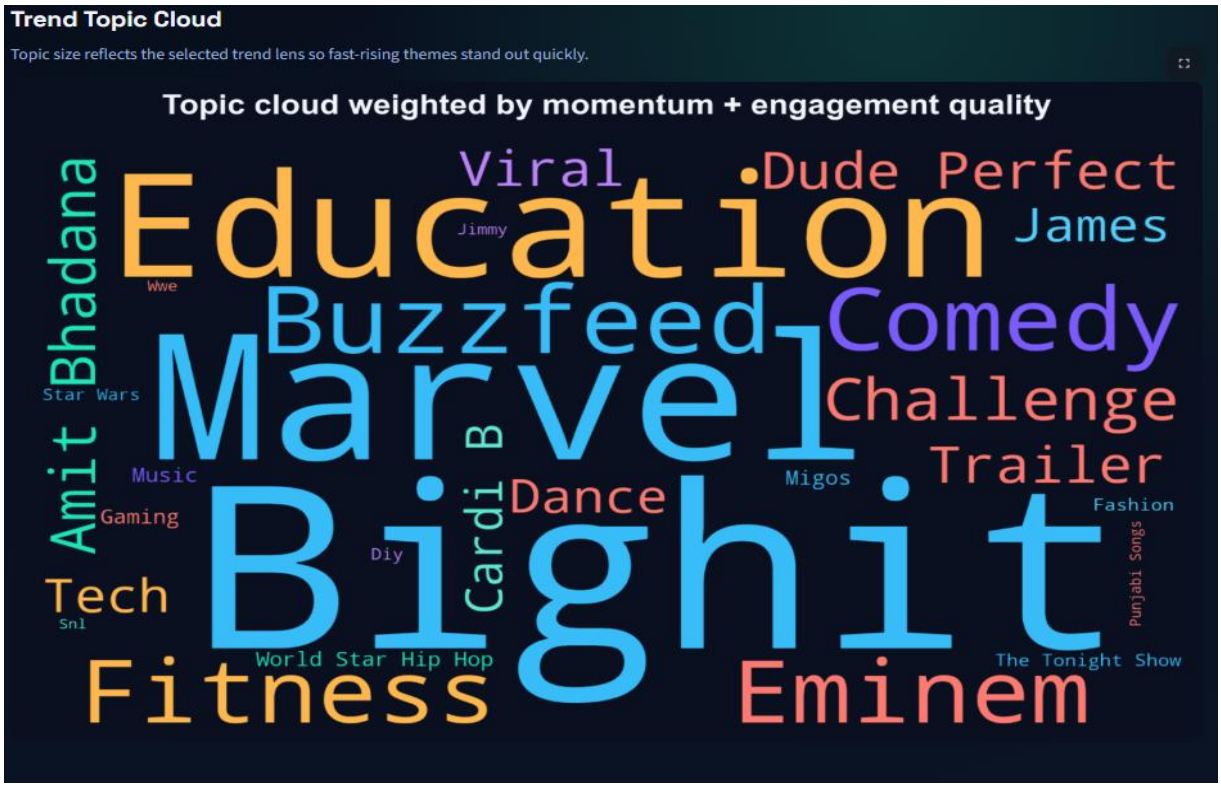

Figure 5 Topic Cloud Weighted by Momentum and Engagement Quality

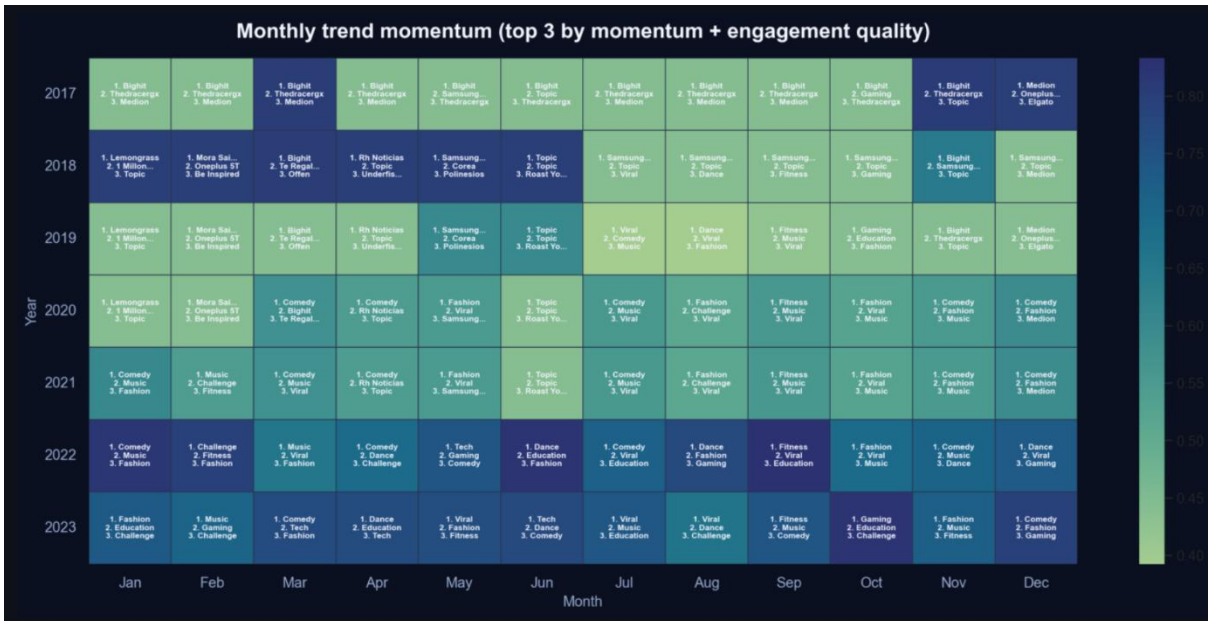

Figure 6 Monthly Trend Momentum Calendar (Top-3 Topics per Month)

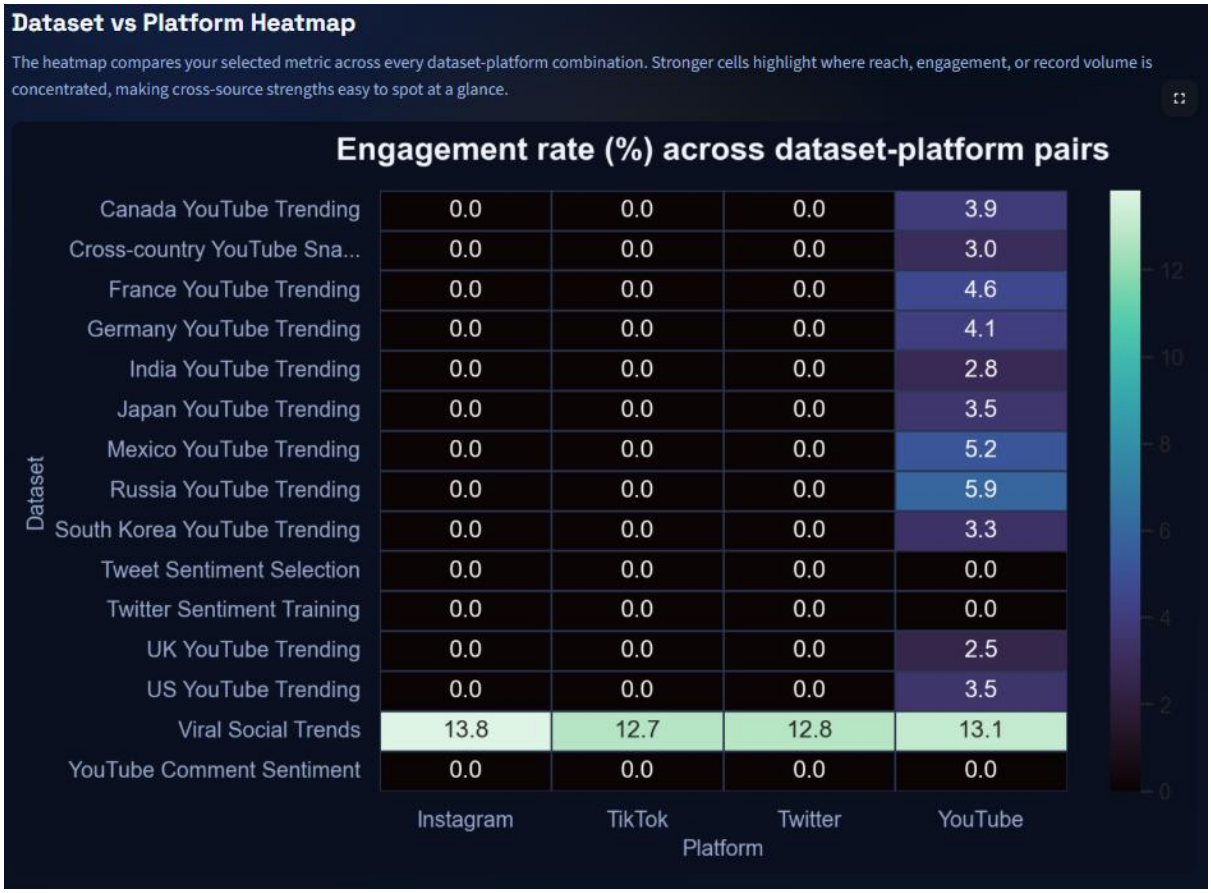

Figure 7 Engagement Rate Heatmap Across Dataset–Platform Combinations

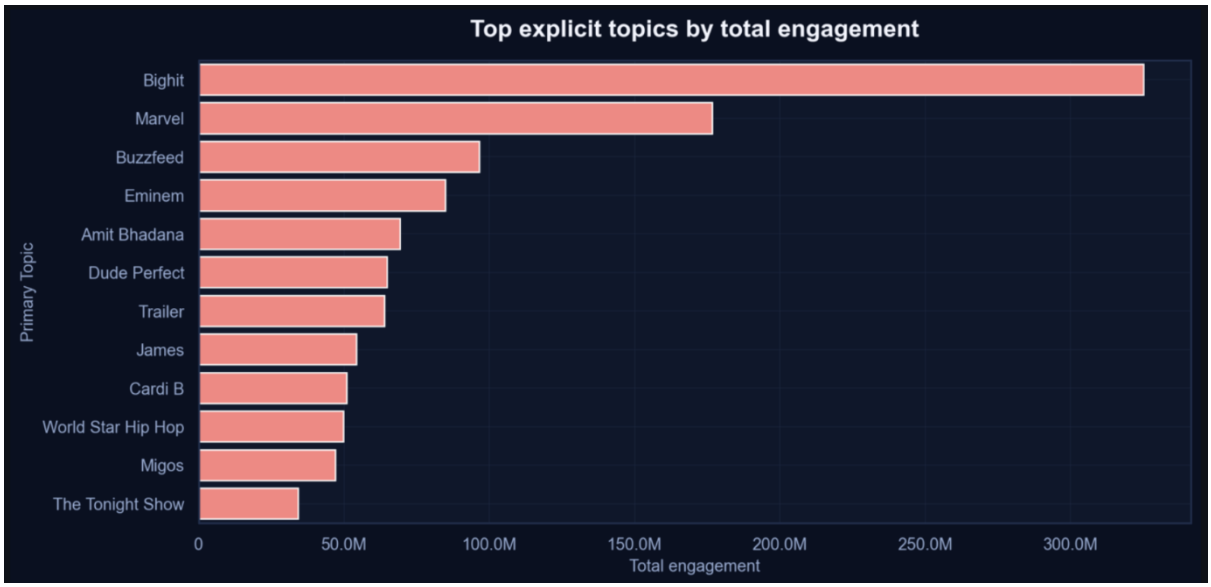

Figure 8 Top Topics Ranked by Total Engagement

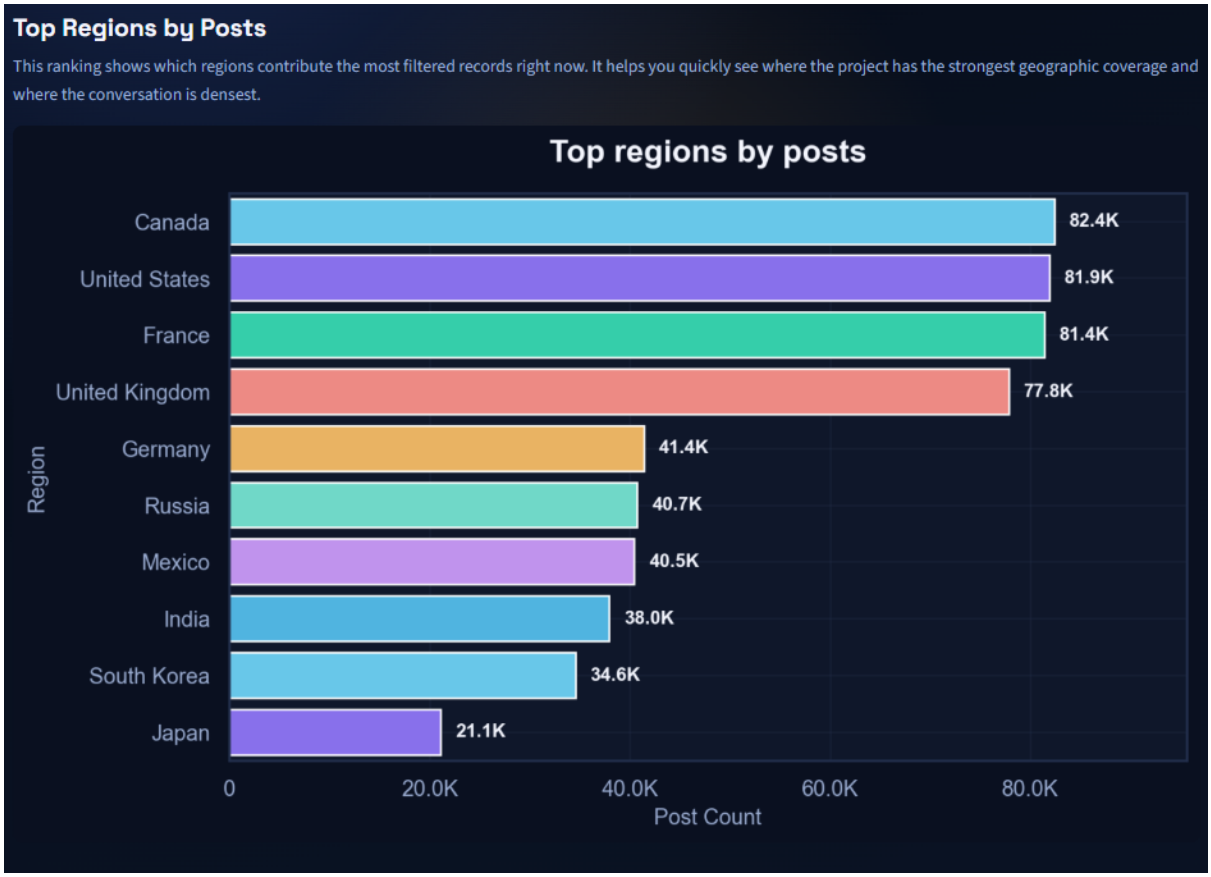

Figure 9 Top Regions Ranked by Post Volume

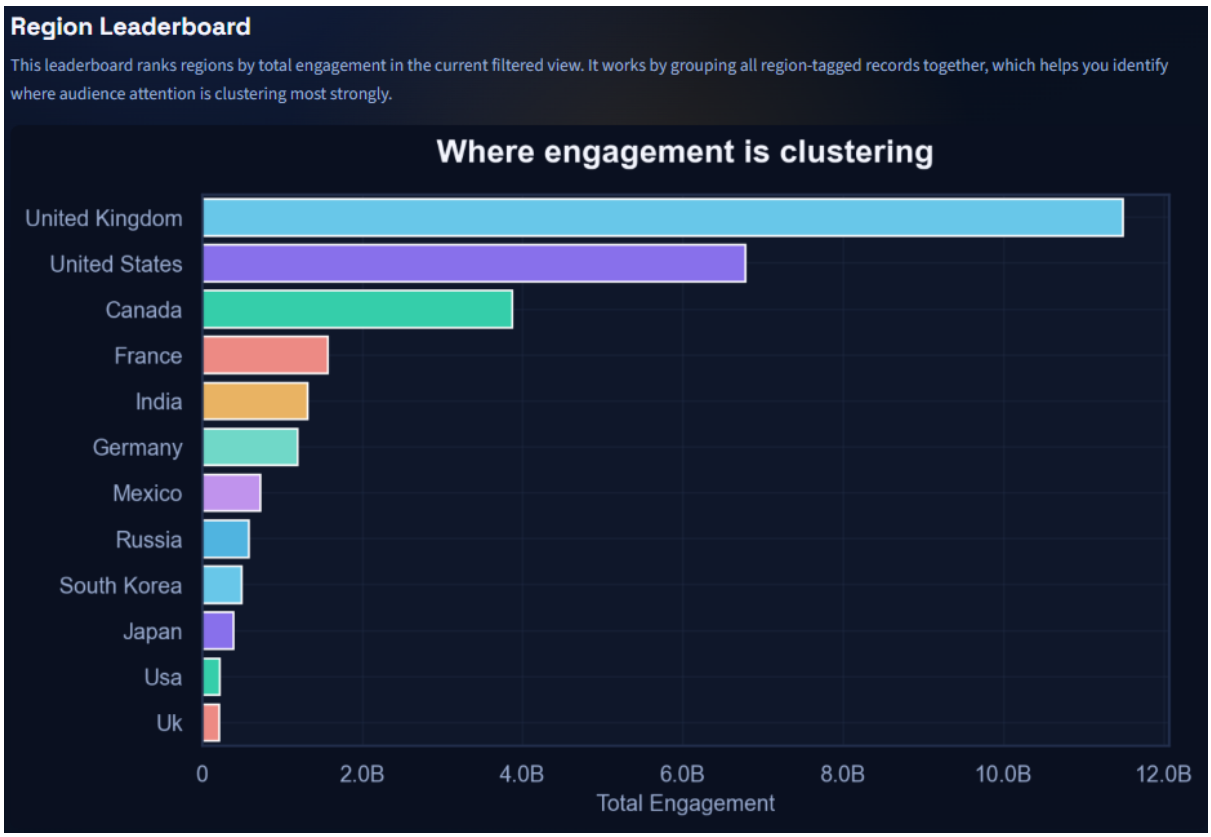

Figure 10 Regional Engagement Leaderboard